

COS214 PRACTICAL 2

U25085892 - Tumisho Matsiu

U25245962 - Shreya Keogh

Task1:

1.1

map is trying to be a composite and it should be a composite.

place features is trying to be a decorator and it should be a decorator.

biomes is trying to be an abstract factory and it should be an abstract factory.

movement is trying to be a strategy design pattern but it should be a state design pattern.

routes is trying to be a state but it should be a strategy design pattern.

movement and routes were the two swapped ones, the reason movement should be state is because the intent of state is to let an object change its behaviour

when its internal state changes, and the states a character can be in terms of movement will each have quite different behaviours. The intent of strategy

is to define a family of algorithms and encapsulate them and make them interchangeable,

which makes sense of routes because each route will still have the same goals but they'll have differing ways of getting to those goals.

1.2

Map: component, leaf, composite client

Place features: component, concrete component, Decorator, concreteDecorator

Biomes: client, AbstractFactory, concreteFactory, AbstractProduct, ConcreteProduct

Movement: State, ConcreteState, context,

Routes: context, strategy, concreteStrategy

1.3

Wrong Pattern Choice:

- State and Strategy are swapped. State is meant to specify how the traveller moves and Strategy determines how routes are chosen. With this current implementation both player movement and route interactions will break.

Wrong UML notation:

- All the nodes have their attributes and operations defined at the same level which is incorrect UML notation for class diagrams.
- The Controller operations move(), plan() and build() do not have specified return types the consequence of this is that when implementing the code the scope of what needs to be defined may be broken and affect other parts of the game.

Broken ownership or missing virtual destructors:

- Composite and Decorator have inconsistent relationships. All the other derived Location classes are correctly inherited while QuestLocation owns the parent class

Location and TollLocation has an association with it. This means that QuestLocation is now responsible for all of Location's memory and the lack of virtual destructors in the base and derived classes will cause memory leaks.

- The dashed arrows in the Abstract Factory section are in the incorrect direction. Their current direction now implies that all Abstract Factory is dependent on all the other subclasses. This means that anytime a WorldBuilder object is created all the other subclasses must be instantiated breaking the variety in biome content generated
- The Controller composes of all the other game features so when it is deallocated it is responsible for the deallocation of the other classes but the lack of destructors breaks this ownership and will cause memory leaks and the game to crash.

Excessive coupling:

- Abstract Factory has all of its factory and product creation coupled into a singular make function. This excessive coupling breaks the Open-Closed Principle as anytime a newly added product and factory are implemented the giant switch statement must be modified which opens the code to new bugs and re-testing of the entire biome module.
- The doMove() function has a large switch statement that couples all the possible movement states into it. This coupling affects the maintainability of the system and creates complicated interactions between the states that depend on another.

1.4

The GameManager class has been made to know about everything, which defeats the coupling goals all the attempted design patterns are trying to implement. This is because GameManager is aware of everything in all the classes which gives too many unrelated responsibilities, or all of the responsibilities available. It also becomes difficult to test each part of the game separately because everything becomes so tightly coupled.

1.5 The overlapping arrows makes it hard to follow the relationships between classes and can lead to confusion on what connections are actually present. To fix this, avoid having arrows overlapping and make them bend to ensure the path they follow is visible as a means to provide clarity to the reader. In addition to that the levels of the nodes also affect how we see the relationships within the diagram; to improve this the classes with child classes should have their children set at lower levels to show the inheritance relationship more clearly and the various associations shown at the same level.

Strategy - RouteStrategy

Concrete strategies - ShortestRouteStrategy, FastestRouteStrategy, ScenicRouteStrategy

Context - Trip

Strategy has been abstracted to have the different route type defined as separate classes with their own algorithmic implementations of the route type functionality and have an interface that allows the Trip context to interact with. To Lessen the coupling between the concrete decorators.

Composite:

Component - Map

Leaf - Destination

Composite - Region

Client - GameManager

In the composite pattern the Map (component) has two child classes that inherit from it, the Destination and Region class. The Map is an abstract component that treats its leaves (Destinations) and compositions (Regions) the same. The client only interacts with the Map, abstracting the complexities of the leaves and compositions.

Decorator:

Component - Map

Concrete component - Destination

Decorator - QuestDestination, WeatherDestination

Concrete Decorators - RainWeatherDestination, SunnyWeatherDestination,

BossQuestDestination, FetchQuestDestination

Client - GameManager

We made two separate abstract decorator classes each to decorate the destination class, QuestDestination and WeatherDestination. This was to divide the possible decorations into 2 categories, the weather or whether or not there is some sort of quest connected to the destination.

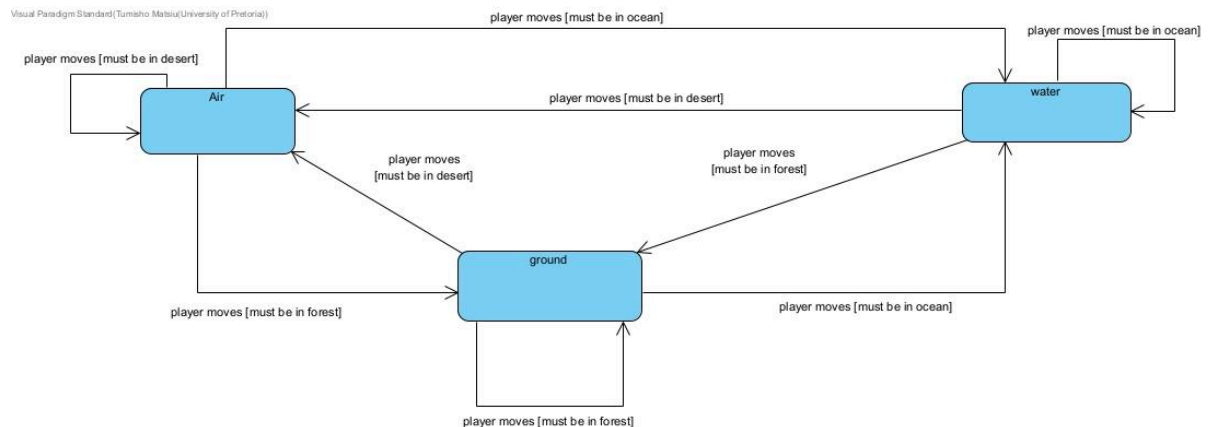
2.3

- ForestFactory and all its corresponding products extend the Abstract Factory design pattern. It slots in without changing any existing classes as there are no switch blocks in our system that determine what kind of product is produced, the creation interaction is simply done through interfacing WorldBuilder.
- The addition of RainWeatherDestination and SunnyWeatherDestination which are derived classes of WeatherDestination. These extend the Decorator design pattern.

These decorations are additive as they are of Map type as well so when created it is essentially just producing new map objects rather than changing the actual internal state and behaviour of the Map object.

- The ObstacleFactory and its corresponding products extend the Abstract Factory design pattern. It slots in without changing any existing classes as there are no switch blocks in our system that determine what kind of obstacle product is produced, the creation interaction is simply done through interfacing WorldBuilder.

Task 3:

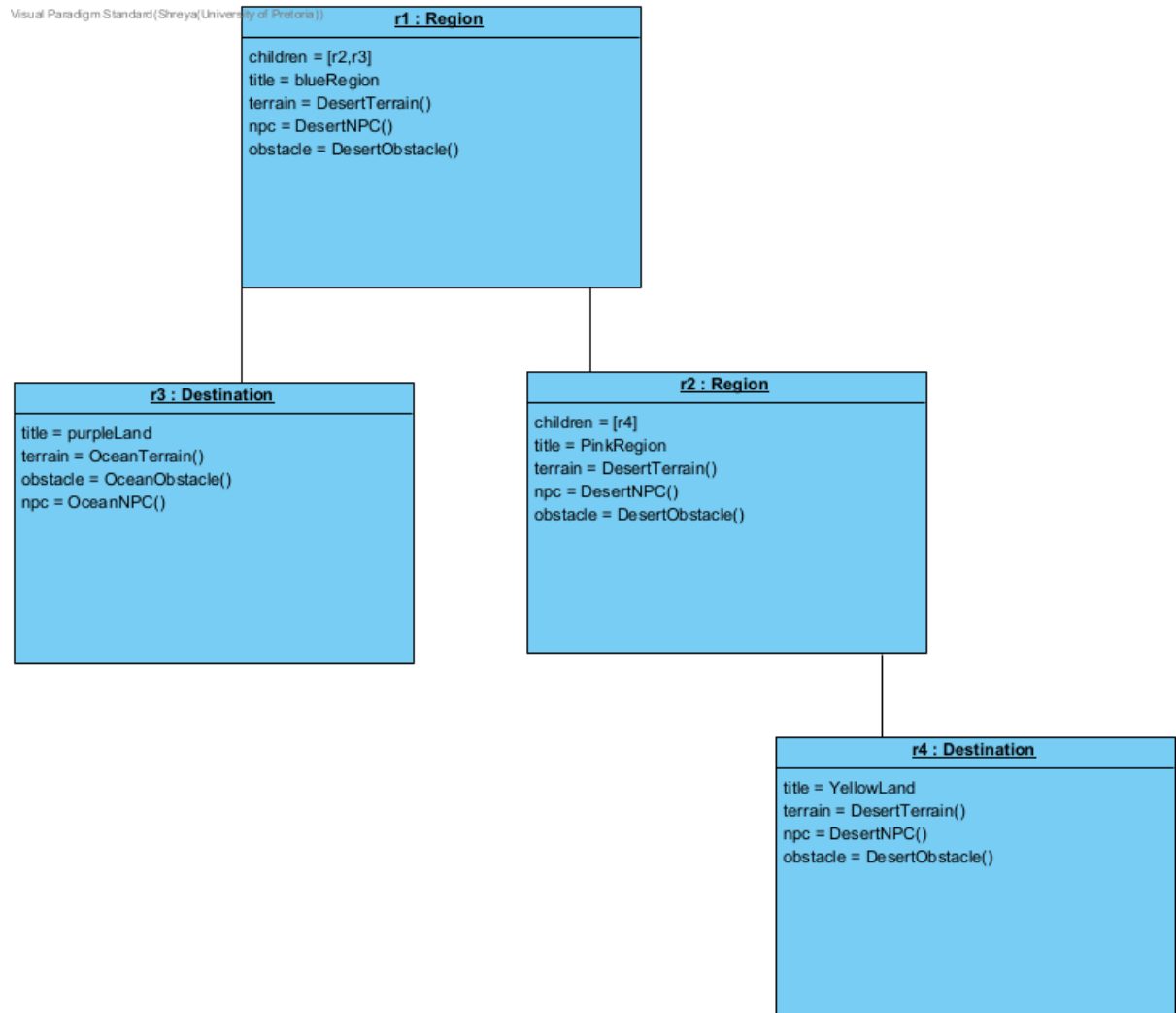


Task 4:

4.3 The intent of state is to allow an object to alter its behaviour when its internal state changes because of this it makes the most sense to make the movement state because depending on the state the player is in, the way that they move will be different. Whereas the intent of Strategy is to define a family of algorithms, encapsulate each one, and make them interchangeable. This makes strategy perfect for the routes because each route has the same goal of leading the player to a certain point, but we can implement different strategies to do that. Because the intents of these patterns are so different, despite their class diagrams looking alike, they are not interchangeable.

Task 5:

5.2



Task 6:

6.2 in the diagram, there should be all the fields mentioned in the objects that the object is wrapping, I just couldn't get it to work in visual paradigm

